Face Special Effects

Face Special Effects
- 1. Content Description
- 2. Program Startup
- 3. Core Code Analysis

1. Content Description

This program implements acquiring the color image and using the dlib library to perform face detection, as well as face special effects. This section requires entering commands in the terminal, where the terminal is opened according to the mainboard type. This lesson uses the Raspberry Pi 5 as an example.

For the Raspberry Pi 5 mainboard, you need to open a terminal on the host machine, then enter the command to access the Docker container, and enter the commands mentioned in this lesson inside the terminal after entering the Docker container. For the tutorial on entering the Docker container from the host, please refer to **【Robot Configuration and Operation Guide】, section 【3.Enter Docker (For Raspberry Pi)】** in this product tutorial.

For the Orin and RDK mainboards, simply open a terminal and enter the commands mentioned in this lesson.

2. Program Startup

The Raspberry Pi PI5 controller needs to enter the Docker container first; RDK X5 and Orin controllers do not need to.

To enter the Docker container (for steps, refer to 【Docker Course】 --- 【4. Docker Startup Script】),

All the docker terminals for the commands below need to enter the same Docker container before executing **(for steps, refer to 【Docker Course】 --- 【3.Docker Submission and Multi-Terminal Access】)**

First, enter the following command in the terminal to start the robot camera and low-level driver environment:

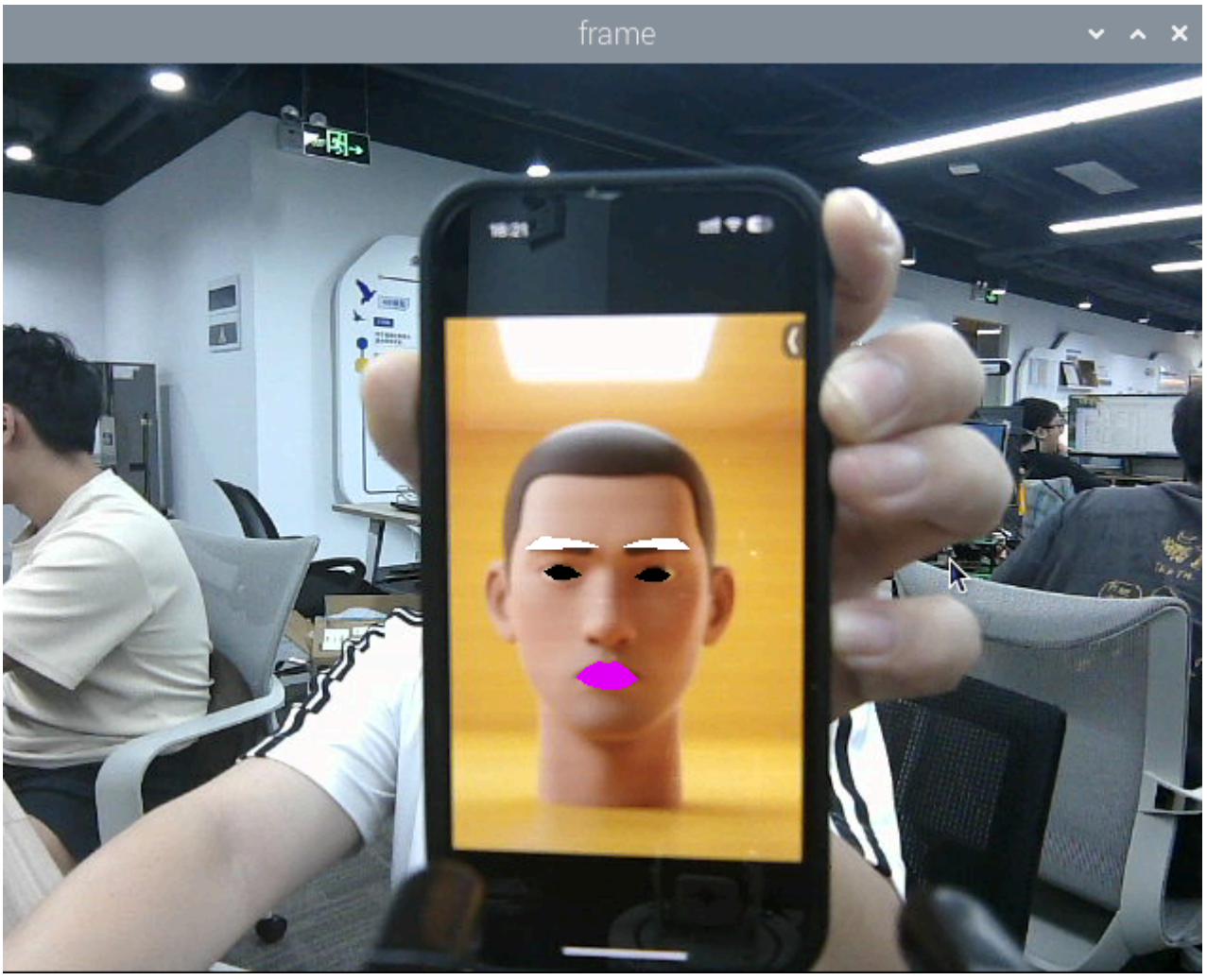
```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

> **Note**
>
> This command is the official all-in-one startup script for the A1Pro, which automatically launches the image stream node according to the camera hardware configuration (USB monocular camera or Nuwa 3D depth camera) and loads the low-level drivers. If you only need to start the camera stream separately, you can also run `ros2 launch usb_cam camera.launch.py` (USB camera) or `ros2 launch ascamera hp60c.launch.py` (Nuwa camera).

After the camera is started successfully, open another terminal and enter the following command in the terminal to start the face special effects program:

```
ros2 run yahboomcar_mediapipe 06_FaceLandmarks
# Control the robotic arm to look forward
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 180,
joint3: 0, joint4: 0, joint5: 90, joint6: 0, time: 1000}" --once
```

After the program runs, as shown in the figure below, it first detects the face, and then applies special effects to the eyebrow, eye, and mouth regions:



3. Core Code Analysis

Program code path:

- Raspberry Pi 5 mainboard

  The program code is located in the running Docker. The path in the Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/06_FaceLandmarks.py`

- Orin mainboard

  The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/06_FaceLandmarks.py`

- RDK mainboard

  The program code path is

`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/06_FaceLandmarks.py`

Import the required library files:

```
import time
import dlib
import os
import cv2 as cv
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import os
```

Introduction to the dlib library:

DLIB is a modern C++ toolkit containing machine learning algorithms and tools that can be used to create complex software in C++ to solve real-world problems. It is widely used in industry and academia in fields such as robotics, embedded devices, mobile phones, and large high-performance computing environments. The dlib library uses 68 landmark points to mark important parts of the face; for example, points 18–22 mark the right eyebrow, and points 51–68 mark the mouth. The `get_frontal_face_detector` module of the dlib library is used to detect faces, and the `shape_predictor_68_face_landmarks.dat` feature data is used to predict facial feature values.

The 68 face landmarks of dlib are arranged in the following order:

- 0-16: Jaw contour
- 17-21: Right eyebrow
- 22-26: Left eyebrow
- 27-35: Nose bridge and nose tip
- 36-41: Right eye
- 42-47: Left eye
- 48-67: Lip contour

Initialize the data and define the Publisher and Subscriber:

```
def __init__(self, dat_file):
    super().__init__('face_landmarks')
    # Import the database
    self.dlib_facelandmark = dlib.shape_predictor(dat_file)
    # Define the face detection object
    self.hog_face_detector = dlib.get_frontal_face_detector()
    self.dlib_facelandmark = dlib.shape_predictor(self.dat_file)

    self.bridge = CvBridge()

    # Define the Subscriber for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
```

```
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type == 'nuwa' else
'/usb_cam/image_raw'

    self.subscription = self.create_subscription(
        Image,
        topic_name,
        self.image_callback,
        10)
```

Color image topic callback function:

```
def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    # Pass the obtained image into the defined get_face function and return the detected
image
    frame = self.get_face(rgb_image, draw=False)
    # Call the prettify_face function to apply special effects to the image
    frame = self.prettify_face(frame, eye=True, lips=True, eyebrow=True, draw=True)
    cv.imshow('Face Landmarks', frame)
```

The get_face function for face detection:

```
def get_face(self, frame, draw=True):
    # Convert the image space from BGR to grayscale for subsequent image processing
    gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
    # Pass the grayscale image to detect the face
    self.faces = self.hog_face_detector(gray)
    for face in self.faces:
        self.face_landmarks = self.dlib_facelandmark(gray, face)
        if draw:
            for n in range(68):
                x = self.face_landmarks.part(n).x
                y = self.face_landmarks.part(n).y
                cv.circle(frame, (x, y), 2, (0, 255, 255), 2)
                cv.putText(frame, str(n), (x, y), cv.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
255), 2)
    return frame
```

The prettify_face function, which applies special effects to the face:

```
def prettify_face(self, frame, eye=True, lips=True, eyebrow=True, draw=True):
    # Eyes
    if eye:
        leftEye = self.get_lmList(frame, 36, 42)
        rightEye = self.get_lmList(frame, 42, 48)
        if draw:
            if len(leftEye) != 0: frame = cv.fillConvexPoly(frame, np.mat(leftEye), (0, 0,
0))
            if len(rightEye) != 0: frame = cv.fillConvexPoly(frame, np.mat(rightEye), (0, 0,
0))
    # Lips
```

```
    if lips:
        lipIndexlistA = [51, 52, 53, 54, 64, 63, 62]
        lipIndexlistB = [48, 49, 50, 51, 62, 61, 60]
        lipsUpA = self.get_lipList(frame, lipIndexlistA, draw=True)
        lipsUpB = self.get_lipList(frame, lipIndexlistB, draw=True)
        lipIndexlistA = [57, 58, 59, 48, 67, 66]
        lipIndexlistB = [54, 55, 56, 57, 66, 65, 64]
        lipsDownA = self.get_lipList(frame, lipIndexlistA, draw=True)
        lipsDownB = self.get_lipList(frame, lipIndexlistB, draw=True)
        if draw:
            if len(lipsUpA) != 0: frame = cv.fillConvexPoly(frame, np.mat(lipsUpA), (249, 0,
226))
            if len(lipsUpB) != 0: frame = cv.fillConvexPoly(frame, np.mat(lipsUpB), (249, 0,
226))
            if len(lipsDownA) != 0: frame = cv.fillConvexPoly(frame, np.mat(lipsDownA),
(249, 0, 226))
            if len(lipsDownB) != 0: frame = cv.fillConvexPoly(frame, np.mat(lipsDownB),
(249, 0, 226))
    # Eyebrows
    if eyebrow:
        lefteyebrow = self.get_lmList(frame, 17, 22)
        righteyebrow = self.get_lmList(frame, 22, 27)
        if draw:
            if len(lefteyebrow) != 0: frame = cv.fillConvexPoly(frame, np.mat(lefteyebrow),
(255, 255, 255))
            if len(righteyebrow) != 0: frame = cv.fillConvexPoly(frame,
np.mat(righteyebrow), (255, 255, 255))
    return frame
```

The get_lmList function for obtaining face coordinates:

```
def get_lmList(self, frame, p1, p2, draw=True):
    # Define an empty list
    lmList = []
    # Determine whether a face has been detected
    if len(self.faces) != 0:
        # Iterate through the face list to obtain the xy coordinates of the points in each
range
        for n in range(p1, p2):
            x = self.face_landmarks.part(n).x
            y = self.face_landmarks.part(n).y
            # Add the point coordinates of the face to the list
            lmList.append([x, y])
            if draw:
                next_point = n + 1
                if n == p2 - 1: next_point = p1
                x2 = self.face_landmarks.part(next_point).x
                y2 = self.face_landmarks.part(next_point).y
                cv.line(frame, (x, y), (x2, y2), (0, 255, 0), 1)
    return lmList
```